ASSIGNMENT 5

Github Developer Explorer

Github API Integration, Async/Await, Redis and IndexedDB Implementation

School of Computing and Information Technology

Spring - 2026

COURSE	ISIT207 Frontend Web Programming
ASSIGNMENT	Assignment 5
PROJECT TYPE	Github Developer Profile Explorer
TECHNOLOGY	HTML5, CSS3, JavaScript, Github API, Async/Await, Redis, IndexedDB
PREPARED BY	Name: Rifat Ahmed

Table of Contents

1. Project Overview
2. Github API Developer Explorer
3. Detailed Profile View
4. Async/Await Implementation
5. Redis Server Integration
6. IndexedDB Developer Storage
7. Requirement Traceability
8. Screenshots
9. Conclusion

1. Project Overview

The Github Developer Explorer website was developed to query and display Github developer profiles. The project integrates Github API communication, asynchronous JavaScript, Redis key-value storage demonstration and IndexedDB browser database operations.

1.1 Assignment Summary Table

The following table summarizes the completed implementation.

Area	Implementation	Files
Github API	Random highlighted developers, profile query and profile details	github-api.js, profile.js
Async/Await	Promise rewritten using async/await	async-await.js
Redis	SET and GET key-value operations	redis-test.js
IndexedDB	Developer storage and deletion functions	indexeddb.js

2. Github API Developer Explorer

The homepage retrieves Github users through the API and displays six randomly selected developer cards. The displayed information includes username, avatar, homepage, location and administrator status. The list automatically updates every 20 seconds.

3. Detailed Profile View

Selecting a username opens a detailed profile page. Additional profile records include name, followers, following, public repositories, creation date and Github profile link.

4. Async/Await Implementation

The Promise based implementation from Assignment 4 was rewritten using async/await. Error handling is managed through try and catch blocks.

5. Redis Server Integration

The project includes a NodeJS Redis client example. It connects to Redis, stores a key-value pair and retrieves the stored value.

6. IndexedDB Developer Storage

A separate database page stores DeveloperID, homepage URL and location. Duplicate IDs are rejected, URLs are validated for HTTP/HTTPS format, and delete operations are available.

7. Requirement Traceability

Implemented requirements: Github API user reading, six highlighted developers, 20-second refresh, detailed profile display, username search, Async/Await conversion, Redis demonstration and IndexedDB CRUD operations.

8. Screenshots

Screenshots of Home, Profile and IndexedDB pages can be inserted following the same presentation style used in Assignment 4.

9. Conclusion

The assignment successfully demonstrates modern frontend development techniques including API integration, asynchronous programming, browser storage and server-side database interaction.